credit_card

May 1, 2026

[131]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings('ignore')
```

[132]:
```
df=pd.read_csv('creditcard_2023.csv')
```

[133]:
```
df.head()
```

[133]:
```
   id        V1        V2        V3        V4        V5        V6        V7  \
0   0 -0.260648 -0.469648  2.496266 -0.083724  0.129681  0.732898  0.519014
1   1  0.985100 -0.356045  0.558056 -0.429654  0.277140  0.428605  0.406466
2   2 -0.260272 -0.949385  1.728538 -0.457986  0.074062  1.419481  0.743511
3   3 -0.152152 -0.508959  1.746840 -1.090178  0.249486  1.143312  0.518269
4   4 -0.206820 -0.165280  1.527053 -0.448293  0.106125  0.530549  0.658849

         V8        V9  …       V21       V22       V23       V24       V25  \
0 -0.130006  0.727159  … -0.110552  0.217606 -0.134794  0.165959  0.126280
1 -0.133118  0.347452  … -0.194936 -0.605761  0.079469 -0.577395  0.190090
2 -0.095576 -0.261297  … -0.005020  0.702906  0.945045 -1.154666 -0.605564
3 -0.065130 -0.205698  … -0.146927 -0.038212 -0.214048 -1.893131  1.003963
4 -0.212660  1.049921  … -0.106984  0.729727 -0.161666  0.312561 -0.414116

        V26       V27       V28    Amount  Class
0 -0.434824 -0.081230 -0.151045  17982.10      0
1  0.296503 -0.248052 -0.064512   6531.37      0
2 -0.312895 -0.300258 -0.244718   2513.54      0
3 -0.515950 -0.165316  0.048424   5384.44      0
4  1.071126  0.023712  0.419117  14278.97      0

[5 rows x 31 columns]
```

[134]:
```
df['Class'].value_counts()
# class balance
```

[134]:
```
Class
0    284315
1    284315
Name: count, dtype: int64
```

[135]:
```
df.shape
```

[135]:
```
(568630, 31)
```

[136]:
```
# EDA
df.isnull().sum()

# No null preset
# from V1 to V28 it represent the transaction amount,time,error etc (mention on
 ↪the dataset discription)
```

[136]:
```
id        0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
```

```
dtype: int64
```

[137]:
```python
df.info()
# No null and all are in numaric form so no need of object encoding
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 568630 entries, 0 to 568629
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   id      568630 non-null  int64
 1   V1      568630 non-null  float64
 2   V2      568630 non-null  float64
 3   V3      568630 non-null  float64
 4   V4      568630 non-null  float64
 5   V5      568630 non-null  float64
 6   V6      568630 non-null  float64
 7   V7      568630 non-null  float64
 8   V8      568630 non-null  float64
 9   V9      568630 non-null  float64
 10  V10     568630 non-null  float64
 11  V11     568630 non-null  float64
 12  V12     568630 non-null  float64
 13  V13     568630 non-null  float64
 14  V14     568630 non-null  float64
 15  V15     568630 non-null  float64
 16  V16     568630 non-null  float64
 17  V17     568630 non-null  float64
 18  V18     568630 non-null  float64
 19  V19     568630 non-null  float64
 20  V20     568630 non-null  float64
 21  V21     568630 non-null  float64
 22  V22     568630 non-null  float64
 23  V23     568630 non-null  float64
 24  V24     568630 non-null  float64
 25  V25     568630 non-null  float64
 26  V26     568630 non-null  float64
 27  V27     568630 non-null  float64
 28  V28     568630 non-null  float64
 29  Amount  568630 non-null  float64
 30  Class   568630 non-null  int64
dtypes: float64(29), int64(2)
memory usage: 134.5 MB
```

[138]:
```python
# check the stats
df.describe()
```

[138]:

	id	V1	V2	V3	V4 \
count	568630.000000	5.686300e+05	5.686300e+05	5.686300e+05	5.686300e+05
mean	284314.500000	-5.638058e-17	-1.319545e-16	-3.518788e-17	-2.879008e-17
std	164149.486122	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00
min	0.000000	-3.495584e+00	-4.996657e+01	-3.183760e+00	-4.951222e+00
25%	142157.250000	-5.652859e-01	-4.866777e-01	-6.492987e-01	-6.560203e-01
50%	284314.500000	-9.363846e-02	-1.358939e-01	3.528579e-04	-7.376152e-02
75%	426471.750000	8.326582e-01	3.435552e-01	6.285380e-01	7.070047e-01
max	568629.000000	2.229046e+00	4.361865e+00	1.412583e+01	3.201536e+00

	V5	V6	V7	V8	V9 \
count	5.686300e+05	5.686300e+05	5.686300e+05	5.686300e+05	5.686300e+05
mean	7.997245e-18	-3.958636e-17	-3.198898e-17	2.109273e-17	3.998623e-17
std	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00
min	-9.952786e+00	-2.111111e+01	-4.351839e+00	-1.075634e+01	-3.751919e+00
25%	-2.934955e-01	-4.458712e-01	-2.835329e-01	-1.922572e-01	-5.687446e-01
50%	8.108788e-02	7.871758e-02	2.333659e-01	-1.145242e-01	9.252647e-02
75%	4.397368e-01	4.977881e-01	5.259548e-01	4.729905e-02	5.592621e-01
max	4.271689e+01	2.616840e+01	2.178730e+02	5.958040e+00	2.027006e+01

	…	V21	V22	V23	V24 \
count	…	5.686300e+05	5.686300e+05	5.686300e+05	5.686300e+05
mean	…	4.758361e-17	3.948640e-18	6.194741e-18	-2.799036e-18
std	…	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00
min	…	-1.938252e+01	-7.734798e+00	-3.029545e+01	-4.067968e+00
25%	…	-1.664408e-01	-4.904892e-01	-2.376289e-01	-6.515801e-01
50%	…	-3.743065e-02	-2.732881e-02	-5.968903e-02	1.590123e-02
75%	…	1.479787e-01	4.638817e-01	1.557153e-01	7.007374e-01
max	…	8.087080e+00	1.263251e+01	3.170763e+01	1.296564e+01

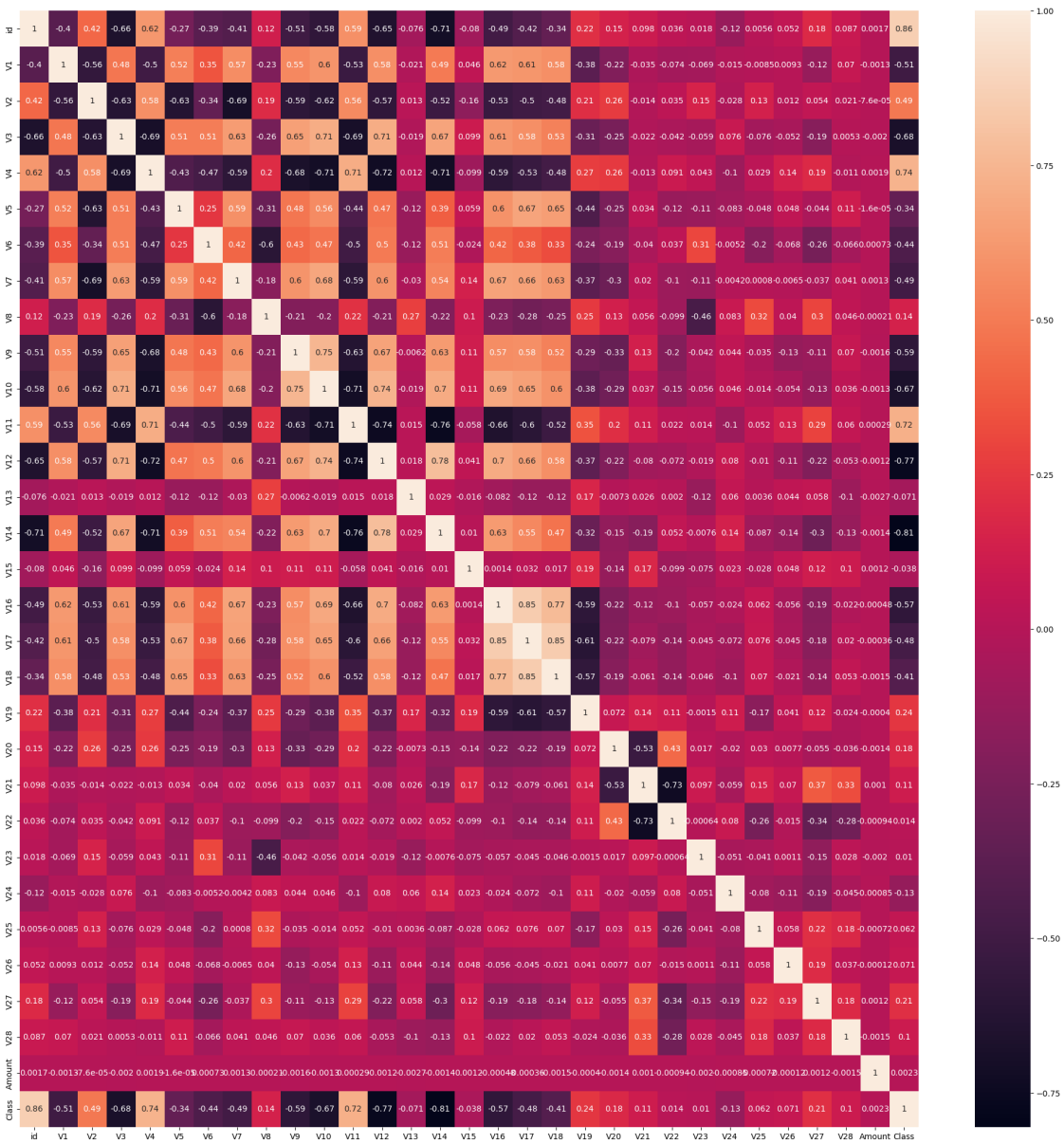
	V25	V26	V27	V28	Amount \
count	5.686300e+05	5.686300e+05	5.686300e+05	5.686300e+05	568630.000000
mean	-3.178905e-17	-7.497417e-18	-3.598760e-17	2.609101e-17	12041.957635
std	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00	6919.644449
min	-1.361263e+01	-8.226969e+00	-1.049863e+01	-3.903524e+01	50.010000
25%	-5.541485e-01	-6.318948e-01	-3.049607e-01	-2.318783e-01	6054.892500
50%	-8.193162e-03	-1.189208e-02	-1.729111e-01	-1.392973e-02	12030.150000
75%	5.500147e-01	6.728879e-01	3.340230e-01	4.095903e-01	18036.330000
max	1.462151e+01	5.623285e+00	1.132311e+02	7.725594e+01	24039.930000

	Class
count	568630.0
mean	0.5
std	0.5
min	0.0
25%	0.0
50%	0.5

```
75%          1.0
max          1.0

[8 rows x 31 columns]
```

[139]:
```python
plt.figure(figsize=(25, 25))
sns.heatmap(df.corr(), annot=True)
plt.show()
```



[141]:
```
feature=df.drop(columns=['Class','id'])
target=df['Class']
```

[142]:
```
from sklearn.preprocessing import StandardScaler
ss=StandardScaler()
feature.iloc[:]=ss.fit_transform(feature.iloc[:])
```

[143]:
```
feature.head()
```

[143]:
```
          V1        V2        V3        V4        V5        V6        V7  \
0 -0.260648 -0.469648  2.496266 -0.083724  0.129681  0.732898  0.519014
1  0.985100 -0.356045  0.558056 -0.429654  0.277140  0.428605  0.406466
2 -0.260272 -0.949385  1.728538 -0.457986  0.074062  1.419481  0.743511
3 -0.152152 -0.508959  1.746840 -1.090178  0.249486  1.143312  0.518269
4 -0.206820 -0.165280  1.527053 -0.448293  0.106125  0.530549  0.658849

          V8        V9       V10  …       V20       V21       V22       V23  \
0 -0.130006  0.727159  0.637735  …  0.091202 -0.110552  0.217606 -0.134794
1 -0.133118  0.347452  0.529808  … -0.233984 -0.194936 -0.605761  0.079469
2 -0.095576 -0.261297  0.690708  …  0.361652 -0.005020  0.702906  0.945045
3 -0.065130 -0.205698  0.575231  … -0.378223 -0.146927 -0.038212 -0.214048
4 -0.212660  1.049921  0.968046  …  0.247237 -0.106984  0.729727 -0.161666

        V24       V25       V26       V27       V28    Amount
0  0.165959  0.126280 -0.434824 -0.081230 -0.151045  0.858447
1 -0.577395  0.190090  0.296503 -0.248052 -0.064512 -0.796369
2 -1.154666 -0.605564 -0.312895 -0.300258 -0.244718 -1.377011
3 -1.893131  1.003963 -0.515950 -0.165316  0.048424 -0.962119
4  0.312561 -0.414116  1.071126  0.023712  0.419117  0.323285

[5 rows x 29 columns]
```

[146]:
```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(feature,target,train_size=0.20)
```

[147]:
```
print(xtrain.shape,ytrain.shape)
print(xtest.shape,ytest.shape)
```

```
(113726, 29) (113726,)
(454904, 29) (454904,)
```

[148]:
```
from sklearn.metrics import classification_report,confusion_matrix
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
```

[149]:
```
def mymodel(model):
    model.fit(xtrain, ytrain)
```

```
    ypred = model.predict(xtest)
    # evaluation metrics
    c = pd.DataFrame(confusion_matrix(ytest, ypred),
                     index = ['Low','High'],
                     columns = ['Low','High'])
    print(f'Confusion Matrix : \n{c}')
    print(f'Classification Report : \n{classification_report(ytest, ypred)}')
    return model
```

[150]:
```
lm=mymodel(LogisticRegression())
lm
```

```
Confusion Matrix :
         Low    High
Low   222703    4812
High   11095  216294
Classification Report :
              precision    recall  f1-score   support

           0       0.95      0.98      0.97    227515
           1       0.98      0.95      0.96    227389

    accuracy                           0.97    454904
   macro avg       0.97      0.97      0.97    454904
weighted avg       0.97      0.97      0.97    454904
```

[150]:
```
LogisticRegression()
```

[151]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout
```

[152]:
```
model = Sequential([
    Dense(128,activation='relu',input_shape=(xtrain.shape[1],)),
    Dense(64,activation='relu'),
    Dense(16,activation='relu'),
    Dense(1,activation='sigmoid')
])
```

[153]:
```
model.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
```

[154]:
```
# train the mdoel
model.fit(xtrain,ytrain,epochs=25,batch_size=100,validation_split=0.20)
```

```
Epoch 1/25
910/910              5s 3ms/step -
accuracy: 0.9726 - loss: 0.0743 - val_accuracy: 0.9907 - val_loss: 0.0262
```

```
Epoch 2/25
910/910              3s 3ms/step -
accuracy: 0.9947 - loss: 0.0173 - val_accuracy: 0.9968 - val_loss: 0.0112
Epoch 3/25
910/910              3s 3ms/step -
accuracy: 0.9976 - loss: 0.0087 - val_accuracy: 0.9975 - val_loss: 0.0090
Epoch 4/25
910/910              3s 3ms/step -
accuracy: 0.9983 - loss: 0.0068 - val_accuracy: 0.9981 - val_loss: 0.0077
Epoch 5/25
910/910              3s 3ms/step -
accuracy: 0.9985 - loss: 0.0058 - val_accuracy: 0.9973 - val_loss: 0.0095
Epoch 6/25
910/910              3s 3ms/step -
accuracy: 0.9987 - loss: 0.0048 - val_accuracy: 0.9980 - val_loss: 0.0078
Epoch 7/25
910/910              2s 3ms/step -
accuracy: 0.9989 - loss: 0.0039 - val_accuracy: 0.9977 - val_loss: 0.0078
Epoch 8/25
910/910              3s 3ms/step -
accuracy: 0.9989 - loss: 0.0037 - val_accuracy: 0.9993 - val_loss: 0.0039
Epoch 9/25
910/910              3s 3ms/step -
accuracy: 0.9991 - loss: 0.0032 - val_accuracy: 0.9981 - val_loss: 0.0078
Epoch 10/25
910/910              3s 3ms/step -
accuracy: 0.9990 - loss: 0.0035 - val_accuracy: 0.9988 - val_loss: 0.0047
Epoch 11/25
910/910              3s 4ms/step -
accuracy: 0.9991 - loss: 0.0032 - val_accuracy: 0.9983 - val_loss: 0.0071
Epoch 12/25
910/910              3s 4ms/step -
accuracy: 0.9993 - loss: 0.0028 - val_accuracy: 0.9986 - val_loss: 0.0060
Epoch 13/25
910/910              3s 3ms/step -
accuracy: 0.9992 - loss: 0.0030 - val_accuracy: 0.9982 - val_loss: 0.0071
Epoch 14/25
910/910              3s 3ms/step -
accuracy: 0.9993 - loss: 0.0021 - val_accuracy: 0.9991 - val_loss: 0.0038
Epoch 15/25
910/910              3s 3ms/step -
accuracy: 0.9995 - loss: 0.0018 - val_accuracy: 0.9989 - val_loss: 0.0041
Epoch 16/25
910/910              3s 3ms/step -
accuracy: 0.9991 - loss: 0.0030 - val_accuracy: 0.9965 - val_loss: 0.0188
Epoch 17/25
910/910              3s 3ms/step -
accuracy: 0.9993 - loss: 0.0023 - val_accuracy: 0.9992 - val_loss: 0.0037
```

```
Epoch 18/25
910/910              3s 3ms/step -
accuracy: 0.9995 - loss: 0.0018 - val_accuracy: 0.9989 - val_loss: 0.0048
Epoch 19/25
910/910              3s 3ms/step -
accuracy: 0.9995 - loss: 0.0018 - val_accuracy: 0.9983 - val_loss: 0.0059
Epoch 20/25
910/910              2s 3ms/step -
accuracy: 0.9995 - loss: 0.0017 - val_accuracy: 0.9990 - val_loss: 0.0042
Epoch 21/25
910/910              2s 3ms/step -
accuracy: 0.9995 - loss: 0.0020 - val_accuracy: 0.9993 - val_loss: 0.0034
Epoch 22/25
910/910              3s 3ms/step -
accuracy: 0.9997 - loss: 0.0012 - val_accuracy: 0.9995 - val_loss: 0.0029
Epoch 23/25
910/910              3s 3ms/step -
accuracy: 0.9993 - loss: 0.0025 - val_accuracy: 0.9983 - val_loss: 0.0054
Epoch 24/25
910/910              3s 3ms/step -
accuracy: 0.9996 - loss: 0.0013 - val_accuracy: 0.9990 - val_loss: 0.0050
Epoch 25/25
910/910              3s 3ms/step -
accuracy: 0.9996 - loss: 0.0020 - val_accuracy: 0.9989 - val_loss: 0.0056
```

[154]:
```
<keras.src.callbacks.history.History at 0x17236584b50>
```

[155]:
```
loss,accuracy=model.evaluate(xtest,ytest)
print(f'test loss {loss}')
print(f'test accuracy {accuracy}')
```

```
14216/14216              24s 2ms/step
- accuracy: 0.9989 - loss: 0.0056
test loss 0.005589163862168789
test accuracy 0.9989228248596191
```

[156]:
```
ypred_prob=model.predict(xtest)
```

```
14216/14216              17s 1ms/step
```

[157]:
```
ypred=(ypred_prob >0.5).astype(int)
```

[158]:
```
from sklearn.metrics import classification_report,confusion_matrix
```

[159]:
```
print(confusion_matrix(ytest,ypred))
print(classification_report(ytest,ypred))
```

```
[[227045    470]
```

```
[    20 227369]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    227515
           1       1.00      1.00      1.00    227389

    accuracy                           1.00    454904
   macro avg       1.00      1.00      1.00    454904
weighted avg       1.00      1.00      1.00    454904
```

[ ]:

```

```